



**HỌC VIỆN
NGÂN HÀNG**
BANKING ACADEMY OF VIETNAM

Web Design

Lecture 9 - JavaScript



6.1. Introduction to JavaScript

6.2. Basic syntax

6.3. Variables and data types

6.4. Operators



What is JavaScript?

- A client-side scripting language → run on the user's machine, relies on the browser's capabilities and settings
- No relation with Java
- Created by Brendan Eich at Netscape in 1995 and originally named "LiveScript."
- Dynamic programming language: the browser reads the code the same way we do and interprets it on the fly, without any compiler
- Loosely typed: we don't need to tell JS what a variable is, just set the value



Whereas the “**structural**” layer of a page is our **HTML** markup, and the “**presentational**” layer of a page is made up of **CSS**, the third “**behavioral**” layer is made up of our **JavaScript**

- Access to all the elements on the web page using DOM
- React to user input: altering either the contents of the page, the CSS styles, or the browser’s behavior on the fly



Whoops! Some errors occurred.

- That username is already in use.
- Email confirmation doesn't match

Username

wilto

Must be at least 4 characters

Email

sample@email.com

Confirm Email

sampel@email.com

Password

Confirm Password

+ Section 1

- Section 2

Collapsible content

+ Section 3

```
<div data-role="popup" id="popupBasic">
  This is a completely basic popup, no options set.
</div>
```



what can javascript do

what can javascript do

what can javascript be used for

what can javascript do for a website

what can javascript programs do



In HTML, JavaScript code is inserted between `<script>` and `</script>` tags.

JavaScript in `<head>`

https://www.w3schools.com/js/tryit.asp?filename=tryjs_where_to_head

JavaScript in `<body>`

https://www.w3schools.com/js/tryit.asp?filename=tryjs_where_to_body

External JavaScript

https://www.w3schools.com/js/tryit.asp?filename=tryjs_where_to_external

To add several script files to one page - use several script tags:

```
<script src="myScript1.js"></script>  
<script src="myScript2.js"></script>
```



Create a simple HTML file containing a `<h1>` shows: ***“This is my first JS example”***

In the HTML file, create a `<button>` shows: ***“Alert”*** and the action when click on it will call **`myFunction()`**

Given the script below, put it into your web page in **3 ways** to see the results

```
<script>
function myFunction() {
    window.alert("Hello world");
}
</script>
```



Placing scripts at the bottom of the `<body>` element *improves the display speed*, because script interpretation slows down the display.

External scripts **cannot contain `<script>`** tags.

Placing scripts in external files has some advantages:

- It separates HTML and code
- It makes HTML and JavaScript easier to read and maintain
- Cached JavaScript files can speed up page loads

An external script can be referenced by ***absolute path*** and ***relative path***



JavaScript Output

JavaScript Statements

JavaScript Syntax

JavaScript Comments



Exercises & Homeworks / Lecture 9 / output.html

- Writing into an HTML element, using innerHTML.
- Writing into the HTML output using document.write().
- Writing into an alert box, using window.alert().
- Writing into the browser console, using console.log().

https://www.w3schools.com/js/js_output.asp

In which situation we apply each of the output type?



Write your name in the output in 4 different ways above

Useful tips:

- Changing the innerHTML property of an HTML element is a common way to display data in HTML.
- Using document.write() after an HTML document is loaded, will delete all existing HTML → The document.write() method should only be used for testing.
- You can skip the window keyword → alert("...").
- Use console.log() for debugging



A script is made up of a ***series of statements***. A statement is a ***command*** that tells the browser what to do.

```
document.getElementById("demo").innerHTML = "Hello Dolly.";
```

The semicolon at the end of the statement tells JavaScript that it's the end of the command, just as a period ends a sentence.

A line break will also trigger the end of a command, but it is a ***best practice to end each statement with a semicolon***.



The JavaScript syntax defines two types of values:

- Fixed values (Literals): numbers or strings
- Variable values

A JavaScript **identifier/name** (*variables and keywords, and functions*) must begin with:

- A letter (A-Z or a-z)
- A dollar sign (\$)
- Or an underscore (_)

JavaScript identifiers are **case-sensitive**: A variable named **myVariable**, a variable named **myvariable**, and a variable named **MYVariable** will be 3 different objects



Single line comments start with `//`.

Multi-line comments start with `/*` and end with `*/`.

When use JavaScript comments:

1. Explain the code
2. Prevent Execution (debugging)



JavaScript Variables

When to Use var, let, or const?

Data types

Arrays



JavaScript Variables can be declared in 4 ways:

- Automatically
- Using var
- Using let
- Using const



1. Always declare variables
2. Always use **const** if the value should not be changed
3. Always use **const** if the type should not be changed (Arrays and Objects)
4. Only use **let** if you can't use const
5. Only use **var** if you MUST support old browsers.
6. You cannot re-declare a variable declared with **let** or **const**, but **var** can



JavaScript can handle many types of data, but for now, just think of **numbers and strings**.

Strings are written inside double or single quotes.
Numbers are written without quotes.

If you put a number in **quotes**, it will be treated as a text string.



An array is a special variable, which can ***hold more than one value***:

```
const cars = ["Saab", "Volvo", "BMW"];
```

If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
let car1 = "Saab";  
let car2 = "Volvo";  
let car3 = "BMW";
```

However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?

The solution is an array!



An array can hold many values under a single name, and you can access the values by referring to an index number.

```
const cars = ["Saab", "Volvo", "BMW"];  
let car = cars[0];
```

Note: [0] is the first element. [1] is the second element.

Changing an Array Element

```
const cars = ["Saab", "Volvo", "BMW"];  
cars[0] = "Opel";
```



Open the **index.html** file in **Lecture 9 - Materials folder**

Run the code in your browser to see the result

Modify the following code:

```
const pi = "3.14";
```

```
let x = pi + 1;
```

```
...
```

```
document.getElementById("demo").innerHTML =
```

```
x + "<br>" + person + "<br>" + answer;
```

Explain the result



Types of JavaScript Operators

JavaScript Arithmetic

JavaScript Assignment



- **Arithmetic Operators**
- **Assignment Operators**
- **Comparison Operators (*self-reading*)**
- **String Operators (*self-reading*)**
- Logical Operators
- Bitwise Operators
- Ternary Operators
- Type Operators



Arithmetic operators perform arithmetic on numbers (literals or variables).

https://www.w3schools.com/js/js_arithmetic.asp

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Remainder)
++	Increment
--	Decrement



Assignment operators assign values to JavaScript variables.

https://www.w3schools.com/js/js_assignment.asp

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y



Viết một đoạn mã JavaScript để thực hiện các bước sau đây:

1. Khai báo một biến có tên là **tuoiHienTai** và gán giá trị bất kỳ là tuổi hiện tại của bạn.
2. Khai báo một biến có tên là **soNamTang** và gán giá trị bất kỳ là số năm bạn muốn tính sau đó.
3. Sử dụng biến **tuoiHienTai** và **soNamTang** để tính tuổi của bạn sau **soNamTang** năm và lưu kết quả vào một biến có tên là **tuoiSauSoNamTang**.
4. Sử dụng toán tử tăng dần (**++**) để tăng giá trị của biến **tuoiHienTai** lên 1 đơn vị.
5. Sử dụng **console.log()** để in ra màn hình thông tin sau **soNamTang** năm và tuổi hiện tại sau tăng thêm 1 năm.



Textbook, page 619